

```
3  struct response *ret;
4  lock_reader(&page_rwlock);
5  ret = get_page();
6  unlock_reader(&page_rwlock);
7  return ret;
8  }
9
10 void update_page(void)
11 {
12     lock_writer(&page_rwlock);
13     // 更新网页逻辑
14     unlock_writer(&page_rwlock);
15 }
```

参考文献

- [1] The first multi-core, 1ghz processor [EB/OL]. [2022-12-06]. <https://www.ibm.com/ibm/history/ibm100/us/en/icons/power4/>.
- [2] Amd shows first dual-core processor [EB/OL]. [2022-12-06]. <https://www.pcworld.com/article/117654/article.html>.
- [3] Dual core era begins, pc makers start selling intel-based pcs [EB/OL]. [2022-12-06]. <https://www.intel.com/pressroom/archive/releases/2005/20050418comp.htm>.
- [4] Arm announces the first mobile multicore processor-cortex-a5 [EB/OL]. [2022-12-06]. https://www.gsmarena.com/arm_announces_the_first_mobile_multicore_processor__cortexa5-news-1200.php.
- [5] RUPP K. 42 years of microprocessor trend data [EB/OL]. [2022-12-06]. <https://www.karlrupp.net/2018/02/42-years-of-microprocessor-trend-data/>.
- [6] Spec benchmarks [EB/OL]. [2022-12-06]. <https://www.spec.org/benchmarks.html>.
- [7] DIJKSTRA E W. Cooperating sequential processes [C] // The Origin of Concurrent Programming. Springer, 1968: 65-138.
- [8] DIJKSTRA E W. Een algorithmen ter voorkoming van de dodelijke omarming [EB/OL]. [2022-12-06]. <http://www.cs.utexas.edu/users/EWD/ewd01xx/EWD108.PDF>.

并发与同步：扫码反馈



同步原语的实现

在第 9 章中，我们介绍了一系列同步原语及其使用方法，包括互斥锁、条件变量、信号量以及读写锁。本章我们将主要关注这些同步原语的实现。在这些同步原语中，有些可以直接在用户态实现，有些则需要操作系统提供一定的辅助才能实现。

10.1 互斥锁的实现

本节主要知识点

- 解决临界区问题的三要素是什么？
- 如何正确实现互斥锁？
- 自旋锁与排号自旋锁的区别是什么？

在第 9 章中，我们介绍了一类最常用的同步原语：互斥锁。互斥锁是为了解决临界区问题而引入的，其能够保证多个线程对于共享资源的互斥访问。在介绍互斥锁的具体实现之前，本节将先介绍临界区问题以及解决该问题的几大要素。

10.1.1 临界区问题

临界区问题中，我们要求同一时刻只能有一个线程执行临界区内的代码（即互斥性）。为了解决临界区问题，我们需要设计一个协议来保证临界区的互斥性。图 10.1 将使用临界区的程序抽象成一个循环，并将这个循环划分成了四个部分。在执行临界区之前，线程必须申请并得到进入临界区的许

```
while (TRUE) {
```

申请进入临界区（获取互斥锁）

临界区部分

标识退出临界区（释放互斥锁）

其他代码

```
}
```

图 10.1 使用临界区的程序

可（即获取互斥锁）。而在退出临界区时，线程也需要显式地标识临界区的结束（即释放互斥锁）。临界区之外的代码不涉及对共享资源的访问，这些代码被归为其他代码。在讨论临界区问题时，我们认为临界区所需的执行时间是有限的，否则该临界区将永久阻塞后续所有需要进入临界区的线程的执行。

在图 10.1 的四个部分中，最为关键的是申请并得到进入临界区的许可与标识退出临界区这两个部分。我们必须针对其设计一套满足以下条件的算法来保证程序的正确性。

- 互斥访问：在同一时刻，最多只有一个线程可以执行临界区。
- 有限等待：线程申请进入临界区之后，必须在有限的时间内获得许可进入临界区，不能无限等待。
- 空闲让进：当没有线程在执行临界区代码时，必须在申请进入临界区的线程中选择一个线程并允许其执行临界区代码，保证程序执行的进展。

下面我们将分别介绍实现互斥锁的不同方法及其适用的场景。

10.1.2 硬件实现：关闭中断

在单核环境中，虽然多个线程不可能并行执行，但调度器可以调度线程在这个核心中交错执行，造成竞争冒险。比如，在第 9 章介绍的计数问题中，即使所有线程运行在同一个处理器核心中，线程还是可能会交错执行，导致丢失更新。

针对单核处理器中存在的竞争冒险，我们可以通过关闭中断来解决临界区问题。在单核中关闭中断意味着当前执行的线程不能被其他线程抢占，因此若在进入临界区之前关闭中断，且在离开临界区时再开启中断，便能够避免当前执行临界区代码的线程被打断，保证任意时刻只有一个线程执行临界区。

那么在单核环境中，关闭中断能否满足解决临界区问题的三个条件（互斥访问、有限等待与空闲让进）？关闭中断可以防止执行临界区的线程被抢占，避免多个线程同时执行临界区，保证了互斥访问。而有限等待依赖于内核中的调度器，如果能保证在有限时间内调度到该线程，则该线程就可以在有限时间内进入临界区，满足有限等待的要求。最后，每个线程在离开临界区时都开启了中断，允许调度器调度到其他线程并执行。因此在某线程执行时如需要进入临界区（意味着没有其他线程在执行临界区，否则中断应关闭且不会轮到当前线程执行），可以关闭中断并执行临界区代码，满足空闲让进的要求。

然而，在多核环境中，关闭中断的方法不再适用。即使同时关闭了所有核心的中断，也不能阻塞其他核心上正在运行的线程继续执行。如果多个同时运行的线程需要执行临界区，关闭中断还是会出现临界区问题。

10.1.3 软件实现：皮特森算法

皮特森算法^[1]是一种通过纯软件手段解决临界区问题的方法，可以解决两个线程

的临界区问题。该算法能够在两个线程同时执行时保证临界区的互斥性，因此可以用在多核 CPU 中解决两个核心上运行的线程同时需要进入临界区的需求。图 10.2 展示了皮特森算法的实现。下面将通过分析代码来说明皮特森算法如何满足解决临界区问题的三个条件。在皮特森算法中，线程 0 与线程 1 进入和退出临界区部分的代码是完全对称的，因此我们主要使用线程 0 作为分析对象，线程 1 的情况与线程 0 类似。我们认为线程 0 与线程 1 执行在两个不同的处理器核心上，因此这两个线程可以同时执行。

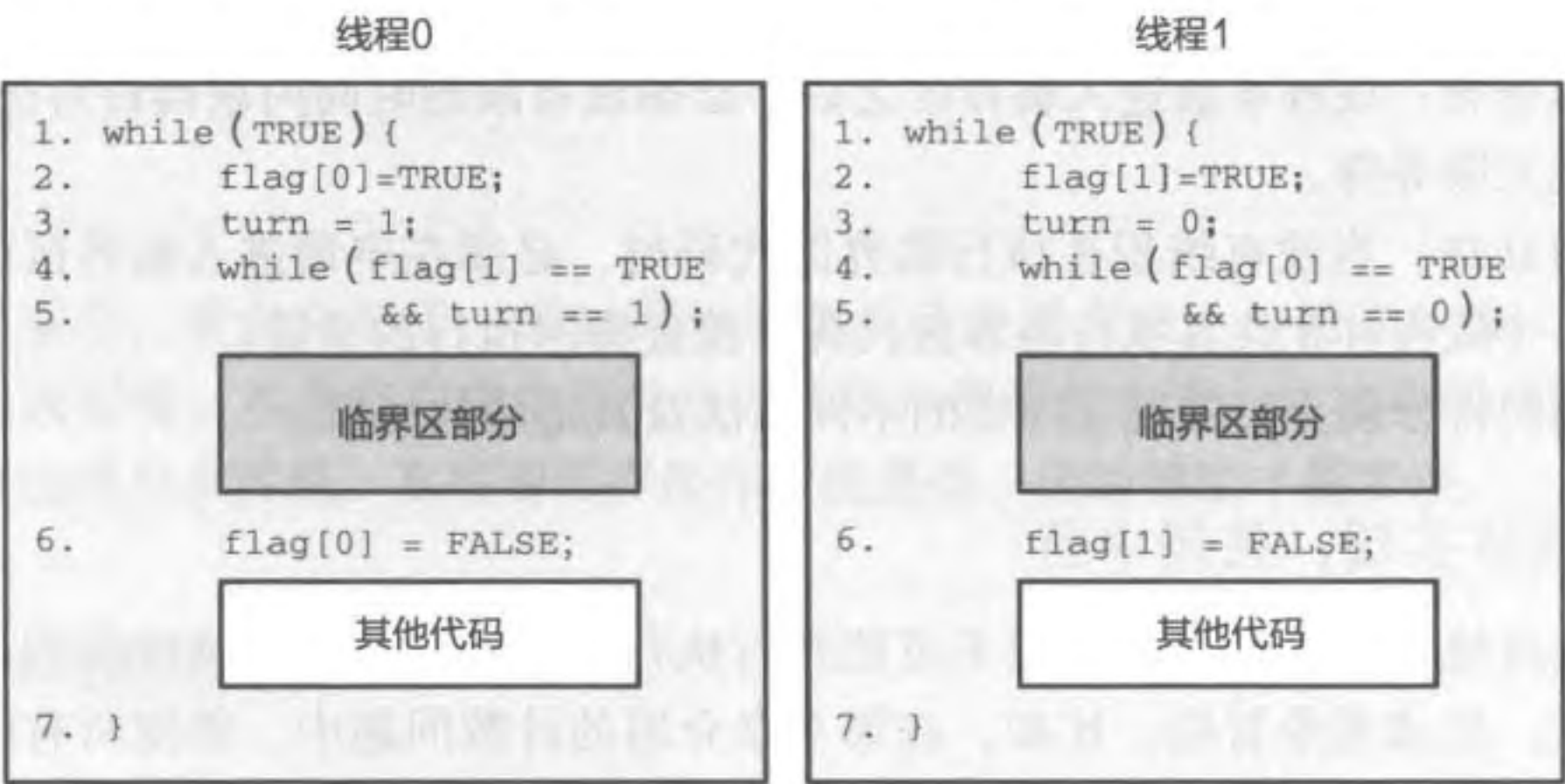


图 10.2 皮特森算法

皮特森算法中有两个重要变量。第一个是全局数组 flag，它共有两个布尔成员 (flag[0] 与 flag[1])，分别代表线程 0 与线程 1 是否正在申请进入临界区。第二个是全局变量 turn。如果两个线程都申请进入临界区，那么 turn 将决定最终能进入临界区的线程编号 (因此其值只能为 0 或 1)。在应用程序开始时，flag 数组中的成员均被设置为 FALSE，表示两个线程都没有申请进入临界区。而变量 turn 无须进行初始化，其会在线程申请进入临界区时被设置。在皮特森算法中，线程在进入临界区之前必须满足以下两个条件之一。如果两个条件均不满足，则需要循环等待，直到其中一个条件得到满足。以线程 0 为例，以下两个条件之一满足之后，方能进入临界区。

- flag[1] == FALSE。表示线程 1 没有申请进入临界区。此时线程 0 可以进入。
- turn == 0。如果 flag[1] == TRUE，即两个线程同时申请进入临界区，此时需要依靠 turn 的值决定最终谁能进入。turn == 0 代表线程 0 可以进入临界区。

如图 10.2 中的代码所示，线程 0 在进入临界区之前，需要设置 flag[0] 为 TRUE，代表其尝试进入临界区。而在线程 0 检查是否可以进入临界区时 (第 4、5 行)，线程 1 只可能处于以下三种状态之一。

- 线程 1 准备进入临界区。由于线程 1 已经执行完第 2 行代码，即标记 flag[1] 为 TRUE，此时两个线程均在申请进入临界区。根据之前的说明，这两个线程将

依据变量 `turn` 的值从二者中选出一个先进入临界区，而另一个将继续循环等待（互斥访问）。`turn` 的值在算法的第 3 行设置，而它的最终值取决于最后对其进行更新的线程。

注意，这里线程都会将 `turn` 设置为对方而不是自己，否则会出现两个线程同时进入临界区的情况。具体而言，线程 1 此时将 `turn` 设置为 0 而非 1，这样可以保证在线程 0 同时申请进入临界区时，线程 1 不能直接进入临界区（满足循环等待的条件，即 `flag[0]==TRUE` 且 `turn==0`）。无论执行的先后顺序如何，`turn` 最终都会选择一个线程进入临界区（空闲让进）。比如，若线程 0 先执行了 `turn = 1` 的操作，而线程 1 后执行 `turn = 0` 的操作，那么 `turn` 的最终值为 0，此时线程 0 可以进入临界区。反之，线程 0 后执行更新操作时，线程 1 可以进入临界区。不存在线程 0 与线程 1 都无法进入临界区的情况。

- 线程 1 在临界区内部。此时由于线程 1 在临界区内，它不会再更新 `flag[1]` 与 `turn`。而由于线程 0 只能将 `turn` 的值设为 1，满足线程 0 循环等待的条件（`flag[1]` 为 `TRUE` 且 `turn` 的值为 1），保证了同一时刻只有一个线程执行临界区内的代码（互斥访问）。
- 线程 1 在执行其他代码（执行完第 6 行代码且执行第 2 行代码之前）。线程 1 在离开临界区时会设置自己的 `flag[1]` 为 `FALSE`，表示不再占有临界区。因此线程 0 可以进入临界区。如果线程 1 在线程 0 成功进入临界区之前再次尝试进入临界区（第 2、3 行），此时由于线程 0 已经进入检查阶段（第 4 行），不会再更新 `turn` 的值，因此 `turn` 的值也会被线程 1 更新为 0，表示线程 0 被允许进入临界区，不会出现无限等待的情况（有限等待）。

通过分析皮特森算法在不同执行情况下的行为，可以发现皮特森算法满足之前定义的解决临界区问题的三个条件。经典的皮特森算法只能应对两个线程的情况，Micha Hofri 于 1990 年扩展了皮特森算法，使其能用于任意数量的线程^[2]。需要注意的是，皮特森算法要求访存操作严格按照程序顺序执行。然而现代 CPU 为了达到更好的性能往往允许访存操作乱序执行，因此皮特森算法无法直接在这些 CPU 上正确工作。为了在这些 CPU 上保证程序的正确性，需要在合适位置添加内存屏障（Memory Barrier）来维护多条访存操作之间的全局可见顺序，从而最终达成与顺序执行一致的效果。由于本章主要关注同步原语，为了方便起见，后续讨论均认为 CPU 严格按照程序顺序执行访存操作^①，且不同的核心也将严格按照程序顺序观察这些访存操作的执行结果。如果读者希望在现代 CPU 中实现这些同步原语，还需要在合适的位置添加内存屏障以保证访存操作的顺序。

① 除了处理器会乱序执行访存操作以外，编译器在生成二进制时也会调整一些访存操作的顺序。本章一并认为最终处理器严格按照程序顺序执行访存操作。

10.1.4 软硬件协同：使用原子操作实现互斥锁

除了软件方法以外，我们还可以利用硬件提供的原子操作（Atomic Operation）设计新的软件算法来解决临界区问题。

原子操作

原子操作指的是不可被打断的一个或一系列指令。即要么这一系列指令都执行完成，要么这一系列指令一条都没有执行，不会出现执行到一半的状态。最常见的原子操作包括比较与置换（Compare-And-Swap, CAS）、拿取并累加（Fetch-And-Add, FAA）等。

代码片段 10.1 展示了 CAS 和 FAA 操作的基本逻辑。CAS 操作比较地址 `addr` 上的值与期望值 `expected` 是否相等，如果相等则将 `addr` 置换为新的值 `new_value`，否则不进行置换。最后，CAS 将返回 `addr` 所存放的旧值。而 FAA 操作则会读取 `addr` 上的旧值，将其加上 `add_value` 后重新存回该地址，最后返回该地址上的旧值。

代码片段 10.1 CAS 与 FAA 操作

```
1 int CAS(int *addr, int expected, int new_value)
2 {
3     int tmp = *addr;
4     if (*addr == expected)
5         *addr = new_value;
6     return tmp;
7 }
8
9 int FAA(int *addr, int add_value)
10 {
11     int tmp = *addr;
12     *addr = tmp + add_value;
13     return tmp;
14 }
```

注意代码片段 10.1 只是用 C 语言来展现操作逻辑，这段代码本身并不具备原子性。以 FAA 操作为例，假设现在有线程 0 与线程 1 两个线程分别执行 `FAA(&var, 1)` 与 `FAA(&var, 2)` 操作，其中全局共享变量 `var` 中存储的初始值为 `x`。如果 FAA 操作是原子的，则当两个线程执行完 FAA 操作后，`var` 中存储的值理应为 `x+3`。考虑这样一种情况：线程 0 执行完第 11 行代码，拿到变量 `var` 上存储的旧值 `x` 后，线程 1 开始 FAA 操作，并一次性执行完第 11 行到第 13 行的代码。此时变量 `var` 的值为 `x+2`。但是由于线程 0 已经完成第 11 行的操作，局部变量 `tmp` 中存放着旧值 `x`，因此在执行完第 12 行的操作时，线程 0 向共享变量 `var` 中写入的值为 `x+1`。在这种情况下，线程 1 的更新丢失，导致程序发生错误。这种现象被称为更新丢失（Lost Update），为了解决这个问题，我们需要硬件辅助保证上述操作的原子性。

为了在硬件层面支持这些原子操作，不同的硬件平台提供了不同的解决方案。在 Intel 平台，应用程序主要是通过使用带 `lock` 前缀的指令来保证操作的原子性。代码

片段 10.2 展示了在 Intel x86-64 平台上使用内联汇编^①实现原子 CAS 操作的代码。这里 `cmpxchg` 指令用于比较与置换指定的地址与值。它先比较地址 `addr` (对应 `%[ptr]`) 中存储的值与寄存器 `%eax` (即 `expected` 的值, 通过 `" +a" (expected)` 指定) 中的值, 如果相等则将 `new_value` (对应 `%[new]`) 存入地址 `addr` 中。否则将 `addr` 中存储的数据读入寄存器 `%eax` 中 (即 `expected`)。通过给这条指令加上 `lock` 前缀, Intel 处理器可以保证上述比较与置换操作的原子性^[3]。

代码片段 10.2 Intel x86-64 架构原子操作的实现

```

1 int atomic_CAS (int *addr, int expected, int new_value)
2 {
3     asm volatile("lock cmpxchg %[new], %[ptr]"
4         : "+a"(expected), [ptr] "+m"(*addr)
5         : [new] "r"(new_value)
6         : "memory");
7     return expected;
8 }

```

不同于 Intel 处理器的实现, ARM 采用 Load-Link/Store-Conditional (LL/SC) 的指令组合^②。在 Load-Link 时, CPU 将使用一个专门的监视器 (Monitor) 记录当前访问的地址。而在 Store-Conditional 时, 当且仅当监视的地址没有被其他核心修改, 才执行存储操作, 否则将返回存储失败。使用 LL/SC 可以方便地实现之前介绍的两种原子操作。以原子 CAS 为例, 在 Load-Link 阶段, 该指令将告诉硬件使用监视器监视 `addr` 这个地址。当发现 `addr` 中存储的值与 `expected` 相等, 因而需要更新 `addr` 中存储的值时, 我们使用 Store-Conditional 来确保这期间没有其他核心更新过 `addr` 存储的值。

代码片段 10.3 展示了如何在 ARM AArch64 架构下实现原子 CAS 操作。其中 `ldxr` 与 `stxr` 分别对应 Load-Link 操作与 Store-Conditional 操作。第 5 行利用 `ldxr` 读取 `addr` (对应 `%2`) 的值并存入 `oldval` (对应 `%w0`) 中, 同时监视 `addr` 这个地址。第 6 行比较 `oldval` 与 `expected` (对应 `%w3`) 是否相等。如果不相等, 则在第 7 行跳到标记 2 处 (第 10 行) 结束该函数, 返回旧值。否则在第 8 行利用 `stxr` 操作将 `new_value` (对应 `%w4`) 存储到地址 `addr` 中。如果有其他核心修改 `addr` 中的值导致存储失败, 其会通过设置寄存器 `%w1` 为 1 来告知。最后, 第 9 行通过检查寄存器 `%w1` 的值来判断是否失败。如果失败, 则回到标记 1 处 (第 5 行) 重新尝试原子 CAS。

代码片段 10.3 ARM AArch64 架构原子操作的实现

```

1 int atomic_CAS (int *addr, int expected, int new_value)
2 {

```

① 内联汇编的基本语法请参考 <https://gcc.gnu.org/onlinedocs/gcc/Using-Assembly-Language-with-C.html>。

② ARMv8.1 支持 LSE (Large System Extensions), 可以使用单条指令完成原子操作。


```

3  int oldval, ret;
4  asm volatile(
5      "1: ldxr      %w0, %2\n"
6      "      cmp      %w0, %w3\n"
7      "      b .ne     2f\n"
8      "      stxr      %w1, %w4, %2\n"
9      "      cbnz      %w1, 1b\n"
10     "2:"
11     : "=&r" (oldval), "=&r" (ret), "+Q" (*addr)
12     : "r" (expected), "r" (new_value)
13     : "memory");
14  return oldval;
15 }

```

使用硬件原子操作实现的互斥锁种类繁多，不同的互斥锁被用于不同的场景，以达到最好的性能表现。本节将介绍两种不同的互斥锁，分别为利用原子 CAS 实现的自旋锁（Spin Lock）与利用原子 FAA 实现的排号自旋锁（Ticket Lock）。

自旋锁

自旋锁利用变量 `lock` 来表示锁的状态，`lock` 为 1 代表已经有人拿锁，而为 0 表示该锁空闲。代码片段 10.4 为自旋锁的 C 语言实现。这里自旋锁的加锁操作是通过原子 CAS 操作实现的。在加锁时，线程会通过 CAS 判断 `lock` 是否空闲（是否 0），如果空闲则上锁（置为 1），否则将一遍一遍重试（第 9 行）。而放锁时，直接将 `lock` 设置为 0 代表其空闲（第 15 行）。由于大部分 64 位 CPU 对于地址对齐的 64 位单一写操作都是原子的^①，因此这里不需要使用额外的硬件指令保护写操作的原子性。由于本章主要关注同步原语，为了方便起见，后续讨论均认为对于 64 位及 64 位以下数据的单一读写操作为原子的。此外，如 10.1.3 节所述，本章假设处理器严格按照程序顺序执行访存操作。因此，这里无须考虑访存操作乱序的问题。自旋锁同样可以通过原子测试并设置（Test-And-Set, TAS）操作实现，因此也被称为 TAS 自旋锁。原子 TAS 操作会检查目标地址的值是否为 0，若为 0 则将其设置为 1。使用原子 TAS 操作实现自旋锁的原理相同，这里不做过多介绍。

代码片段 10.4 自旋锁

```

1 void lock_init(int *lock)
2 {
3     // 初始化自旋锁
4     *lock = 0;
5 }
6

```

① 不过在书写可移植代码时，还是需要使用语言提供的原子操作（如 `atomic_store`）来保证写操作的原子性，交给编译器决定对于特定硬件是否还需要生成额外的指令。


```
7 void lock(int *lock)
8 {
9     while(atomic_CAS(lock, 0, 1) != 0)
10         ; // 循环忙等
11 }
12
13 void unlock(int *lock)
14 {
15     *lock = 0;
16 }
```

小思考

自旋锁是否满足 10.1.1 节列举的解决临界区问题的三个条件（即互斥访问、有限等待与空闲让进）？

自旋锁的实现简单易懂，下面将具体分析自旋锁是否满足解决临界区问题的三个条件。申请进入临界区的线程会尝试将 lock 的值从 0 改为 1，但是由于我们使用了原子 CAS 操作，因此当锁空闲时，多个竞争者中只有一个成功完成 CAS 操作并获取锁，从而进入临界区（互斥访问与空闲让进）。而当锁已经被持有时，后续线程都将无法成功通过 CAS 操作获取锁，从而无法进入临界区（互斥访问）。然而，自旋锁并不能保证有限等待。这是因为自旋锁不具有公平性。从自旋锁的实现可以看出，自旋锁并非按照申请的顺序决定下一个获取锁的竞争者，而是让所有的竞争者均同时尝试完成原子操作，成功完成原子操作的竞争者就成了锁的持有者。原子操作的成功与否完全取决于硬件特性，因此在一些极端情况下，有些锁的竞争者可能一直无法获取互斥锁，不能保证有限等待。不过，由于自旋锁实现简单，在竞争程度低时非常高效，因此依然广泛应用在各种软件中。

练习

- 10.1 自旋锁放锁操作中为何不需要使用 atomic_CAS 操作？
- 10.2 try_lock 操作在互斥锁空闲时将直接获取互斥锁，而不空闲时会直接返回。这里假设返回值为 0 定义为成功获取互斥锁，返回值为 -1 表示互斥锁不空闲。请写出自旋锁的 try_lock 实现。

排号自旋锁

不同于上一小节介绍的自旋锁，排号自旋锁（Ticket Lock，下文简称为排号锁）采取

了一种更加公平的选择策略：按照锁竞争者申请锁的顺序传递锁。与现实生活中去银行、餐厅等取号排队相似，排号锁的竞争者将依照申请的先后顺序分得一个排队的序号，排号锁将依照序号将锁传递给最先到达的竞争者。因此可以认为，锁的竞争者组成了一个先进先出（First In First Out, FIFO）的等待队列。

代码片段 10.5 给出了排号锁的实现^①。排号锁的结构体有两个成员，其中 owner 表示当前的锁持有者序号，而 next 表示下一个需要分发的序号。获取排号锁需要先通过原子 atomic_FAA 操作拿到最新的序号，同时增加锁的分发序号（第 16 行），从而避免其他竞争者拿到相同的序号。拿到序号后，竞争者将通过判断 owner 的值，等待排到自己的序号（第 17 行）。一旦两者相等，竞争者拥有该锁并被允许进入临界区。而在释放锁时，锁持有者通过更新 owner 的值来将锁传递给下一个竞争者（第 24 行）。由于锁的传递顺序是依照申请锁（执行第 16 行代码）的顺序而定，因此该锁保证了获取锁的公平性，且锁的竞争者均会在有限时间内拿到锁并进入临界区（有限等待）。

代码片段 10.5 排号锁

```
1 struct lock {
2     volatile int owner;
3     volatile int next;
4 };
5
6 void lock_init(struct lock *lock)
7 {
8     // 初始化排号锁
9     lock->owner = 0;
10    lock->next = 0;
11 }
12
13 void lock(struct lock *lock)
14 {
15     // 拿取自己的序号
16     int my_ticket = atomic_FAA(&lock->next, 1);
17     while (lock->owner != my_ticket)
18         ; // 循环忙等
19 }
20
21 void unlock(struct lock *lock)
22 {
23     // 传递给下一位竞争者
24     lock->owner++;
25 }
```

① 为方便起见，这里不考虑整型溢出的情况。可能会存在特殊情况，使得 next 的溢出刚好与 owner 一致，从而导致错误。

10.2 条件变量的实现

本节主要知识点

- 为了支持条件变量，操作系统内核应该提供什么样的能力？
- 如何正确实现条件变量？

条件变量提供了一套睡眠/唤醒机制，当某个条件没有得到满足时，其将挂起调用 `cond_wait` 的线程。另一个线程则可以通过 `cond_signal` 接口唤醒所有挂起的线程。由于将一个线程挂起需要操作系统调度器进行协作，因此条件变量的实现需要操作系统的支持。

条件变量的实现需要考虑并发环境下的正确性。代码片段 10.6 展示了条件变量的一种实现。每一个条件变量的结构体中包含了一个等待队列 `wait_list`，用于记录等待在该条件变量上的线程。针对该等待队列，我们使用了三个队列相关的接口：`list_append` 向队尾添加新的成员，`list_empty` 用于判断队列是否为空，`list_remove` 用于从队首移除一个成员。这些接口都是线程安全的，也即这些操作都是互斥的。因此这里没有用额外的机制来保证等待队列。

代码片段 10.6 条件变量的一种实现

```
1 struct cond {
2     struct thread_list *wait_list;
3 };
4
5 void cond_wait(struct cond *cond, struct lock *mutex)
6 {
7     list_append(cond->wait_list, thread_self());
8     atomic_block_unlock(mutex);      // 原子挂起并放锁
9     lock(mutex);                     // 重新获得互斥锁
10 }
11
12 void cond_signal(struct cond *cond)
13 {
14     if (!list_empty(cond->wait_list))
15         wakeup(list_remove(cond->wait_list));
16 }
17
18 void cond_broadcast(struct cond *cond)
19 {
20     while(!list_empty(cond->wait_list))
21         wakeup(list_remove(cond->wait_list));
22 }
```

线程在调用 `cond_wait` 挂起自己时，需要完成以下两个操作：首先将当前线程加

入等待队列（第 7 行），然后原子地挂起当前线程并放锁（第 8 行）。这两个步骤中第二步最为关键，必须保证原子地完成挂起与放锁。假设不保证原子挂起与放锁可能面临什么问题呢？如果我们先放锁再挂起当前线程，则有可能存在其他线程在放锁与挂起的间隙调用 `cond_signal` 操作。如表 10.1 所示，在 T_0 时刻，线程 0 先释放了互斥锁。因此线程 1 可以在 T_1 时刻获取互斥锁并在 T_2 时刻调用 `cond_signal`。由于线程 0 还未挂起，`cond_signal` 操作无法唤醒该线程，也无法阻拦该线程挂起。最终导致在 T_3 时刻，线程 0 挂起。因此线程 1 的本次唤醒被丢失了，导致出现错误。我们称这种现象为丢失唤醒。若我们先挂起再放锁，被挂起的线程便无法成功放锁，后续线程也不能进入临界区唤醒挂起线程，同样会导致错误。为了避免丢失唤醒，这里使用 `atomic_block_unlock` 操作来原子地完成挂起与放锁操作，而这个操作应当由操作系统辅助完成。我们将在 10.5 节详细介绍 Linux 内核中如何利用 `futex` 机制实现该功能。当该线程被其他线程唤醒之后，应当再次获取锁进入临界区（第 9 行）并返回。

表 10.1 先放锁再阻塞：丢失唤醒

时刻	线程 0	线程 1
T_0	<code>unlock(mutex)</code>	
T_1		<code>lock(mutex)</code>
T_2		<code>cond_signal</code>
T_3	阻塞当前线程	

对应地，`cond_signal` 用于唤醒一个等待在条件变量上的线程。该函数先判断是否有线程等待在条件变量上（第 14 行）。如果存在这样的线程，则从等待队列中移除该线程并利用操作系统提供的唤醒服务（`wakeup`）将该线程唤醒（第 15 行）。

除了 `cond_wait` 与 `cond_signal`，条件变量一般还提供广播（`cond_broadcast`）操作，用于唤醒所有等待在条件变量上的线程。其实现也非常的直观。如代码片段所示，`cond_broadcast` 通过重复判断等待队列是否为空（第 20 行），将等待队列里所有的线程都移出等待队列并一一唤醒（第 21 行）。

10.3 信号量的实现

本节主要知识点

- ❑ 如何实现非阻塞信号量？是否需要操作系统协助？
- ❑ 如何高效实现阻塞信号量？

信号量提供了简单易用的接口以管理资源。`wait` 操作用于消耗资源，当没有可用资源时，其将阻塞并等待。`signal` 操作用于生产资源，其还将唤醒等待在该资源上的线程。第 9 章为了说明信号量两个接口的语义，提供了一个简单信号量实现（代码片段 9.15）。本质上可以将信号量理解成一个计数器：`wait` 操作减少计数器的值而

signal 操作增加计数器的值。然而，这段代码无法正确工作，其存在如下问题：

- 由于可能存在多个线程同时更新共享信号量的情况，如果不使用原子操作对其进行更新，就会出现 10.1.4 节介绍的更新丢失问题。
- 即使使用原子操作来更新信号量（替换为代码片段 10.7），仍然存在正确性问题。考虑这样一个场景，系统中有 3 个线程使用一个共享信号量 *s* 同步，假设当前信号量 *s* 的值为 0，线程 0 与线程 1 使用 wait 操作等在该信号量上（第 3 行）。此时线程 2 调用 signal 释放了一个资源，信号量 *s* 的值被更新为 1。线程 0 与线程 1 将同时离开第 3 行的 while 循环，来到第 5 行利用原子操作对信号量 *s* 进行减 1 操作。信号量 *s* 将被更新为 -1，出现错误（资源不可能为负数）。
- 只有一个资源被释放时，同时通知多个线程明显是不明智的策略。在“僧多粥少”之时，让多余的线程挂起并放弃 CPU 才是明智的选择。

代码片段 10.7 替换成原子操作的信号量实现（错误实现）

```
1 void wait(int *S)
2 {
3     while(*S <= 0)
4         ; // 循环忙等
5     atomic_FAA(S, -1);
6 }
7
8 void signal(int *S)
9 {
10    atomic_FAA(S, 1);
11 }
```

10.3.1 非阻塞信号量

本小节将先介绍信号量的非阻塞实现。非阻塞（Non-Blocking）是指不挂起等待线程，而是直接返回或使用循环忙等进行等待。使用非阻塞信号量的线程在发现信号量不可用时，会一直循环忙等直到信号量可用。如上一小节的分析，代码片段 10.7 主要存在的问题在于：当线程调用 signal 释放一个资源后，多个等待线程可能同时使用原子操作（第 5 行）对资源计数 *s* 进行原子自减，最终减为负数并造成错误。为了解决这个问题，最为直观的方式是使用互斥锁确保同一时刻只有一个等待线程可以获取互斥锁，检查计数器此时大于零并消耗计数器，从而确保不会出现多个线程同时发现计数器大于零并尝试消耗信号量的情况。

代码片段 10.8 展示了这样一种实现。与代码片段 10.7 相比，这里使用互斥锁 `sem_lock` 来保护该信号量。当线程等待信号量时，等待线程将重复释放并获取互斥锁（第 10、11 行）。第 11 行的获取互斥锁操作能够确保离开循环时（即 `value` 大于 0），只有

当前线程可以对信号量计数器进行自减操作（第 13 行）。即使有多个线程同时等待在同一信号量上，一旦某一线程释放了该信号量，只有一个成功获取互斥锁的线程可以运行第 9 行的判断操作，在发现信号量计数器不为 0 后退出循环并拿取该信号量（第 13 行），最后释放互斥锁（第 14 行）。其他的等待线程在成功拿到互斥锁后，会在第 9 行的判断操作中发现之前释放的信号量已经被拿取，因此再次陷入等待。

代码片段 10.8 非阻塞信号量的实现

```
1 struct sem {
2     volatile int value;
3     struct lock sem_lock;
4 };
5
6 void wait(sem_t *S)
7 {
8     lock(&S->sem_lock);
9     while(&S->value <= 0) {
10         unlock(&S->sem_lock);
11         lock(&S->sem_lock);
12     }
13     S->value--;
14     unlock(&S->sem_lock);
15 }
16
17 void signal(sem_t *S)
18 {
19     lock(&S->sem_lock);
20     S->value++;
21     unlock(&S->sem_lock);
22 }
```

该实现虽然能够保证信号量的正确性，但要求所有等待的线程反复获取互斥锁来检查是否有空闲的资源，造成了一定的处理器资源浪费。下面我们将介绍阻塞信号量，其将挂起等待的线程，避免循环忙等造成处理器资源浪费。

10.3.2 阻塞信号量

由于我们需要挂起等待的线程直到有空闲的信号量资源，因此可以很直观地想到使用上一节介绍的条件变量。对于等待信号量的线程，其等待的条件是有空闲的信号量资源；而对于释放信号量的线程，其需要唤醒那些等待在该信号量上的线程。

代码片段 10.9 展示了使用此思路实现的一种阻塞信号量。其中信号量结构体包含三个成员，除了 `value` 与互斥锁 `sem_lock` 之外，还加入了用于挂起与唤醒线程的条件变量 `sem_cond`。与非阻塞的信号量实现（代码片段 10.8）相比，这里将等待线程等待循环内放锁与拿锁的部分替换为等待该信号量对应的条件变量（第 11 行）。其调用 `cond_wait` 挂起该等待线程，直到有其他的线程唤醒。由于 `cond_wait` 也同时拥有

挂起并释放互斥锁、唤醒后重新获取锁的语义（详见 10.2 节），该操作的最终效果与非阻塞实现（不断放锁并拿锁）一致，确保了信号量实现的正确性。对应地，当有线程释放信号量时，除了需要增加信号量计数器（第 20 行）外，还需要调用 `cond_signal` 来唤醒一个挂起等待的线程，让其能够获取刚刚释放的资源。

代码片段 10.9 阻塞信号量的实现 1

```
1 struct sem {
2     volatile int value;
3     struct lock sem_lock;
4     struct cond sem_cond;
5 };
6
7 void wait(sem_t *S)
8 {
9     lock(&S->sem_lock);
10    while(S->value == 0) {
11        cond_wait(&S->sem_cond, &S->sem_lock);
12    }
13    S->value--;
14    unlock(&S->sem_lock);
15 }
16
17 void signal(sem_t *S)
18 {
19     lock(&S->sem_lock);
20     S->value++;
21     cond_signal(&S->sem_cond);
22     unlock(&S->sem_lock);
23 }
```

然而，该实现还存在一个问题：即使当前没有任何线程在等待该信号量，调用 `signal` 的线程仍然需要使用 `cond_signal`，这将会影响信号量的性能。为了避免这个问题，代码片段 10.10 给出了阻塞信号量的另一种实现。不同于之前的实现，这里 `value` 的值既可为正数，也可为负数：当没有线程等待时，`value` 为正数或零，表示剩余的资源数量；而当有线程等待时，`value` 会被减为负数。此时，需要引入变量 `wakeup` 来表示有线程等待时的可用资源数量，其只能为正数或零。该值同时也代表应当唤醒的线程数量，因此取名为 `wakeup`。通过这套机制，线程可以仅在有挂起等待信号量的线程时调用 `cond_signal`，避免了无用的条件变量唤醒操作。在该实现中，如果想判断当前可用的资源数量，应当首先查看 `value`，若 `value` 为正数或零，则取 `value` 为当前资源数量。如果当前还存在还未被成功唤醒等待的线程，则此时 `wakeup` 也会大于 0。所以当前可用的总资源数量为 `value+wakeup`，不过其中数量为 `wakeup` 的资源已经预留给此前等待的线程。否则，如果 `value` 为负数，则表示当前存在等待线程，而 `wakeup` 中保存着预留给这些等待线程但还未被取走的资源的数量。表 10.2 通过

一个具体的示例展示了信号量中 value 与 wakeup 的值在具体场景下如何变动。

代码片段 10.10 阻塞信号量的实现 2

```
1 struct sem {
2     volatile int value;
3     volatile int wakeup;
4     struct lock sem_lock;
5     struct cond sem_cond;
6 };
7
8 void wait(struct sem *S)
9 {
10     lock(&S->sem_lock);
11     S->value--;
12     if (S->value < 0) {
13         do {
14             cond_wait(&S->sem_cond, &S->sem_lock);
15         } while (S->wakeup == 0);
16         S->wakeup--;
17     }
18     unlock(&S->sem_lock);
19 }
20
21 void signal(struct sem *S)
22 {
23     lock(&S->sem_lock);
24     S->value++;
25     if (S->value <= 0) {
26         S->wakeup++;
27         cond_signal(&S->sem_cond);
28     }
29     unlock(&S->sem_lock);
30 }
```

表 10.2 信号量的使用示例

操作（按时间排序）	持有线程	等待线程	value	wakeup	剩余资源	注 释
初始状态	0	0	10	0	10	初始有 10 个可用资源
10 个线程获取资源	10	0	0	0	0	
10 个线程请求资源	10	10	-10	0	0	value 为负数代表有线程在等待
5 个线程释放资源	5	10	-5	5	5	wakeup 为 5 代表其中 5 个资源已经预留给正在等待的线程
1 个等待线程获取资源	6	9	-5	4	4	
4 个等待线程获取资源	10	5	-5	0	0	
5 个线程释放资源	5	5	0	5	5	
1 个等待线程获取资源	6	4	0	4	4	
1 个等待线程获取资源	7	3	0	3	3	
2 个线程释放资源	5	3	2	3	5	5 个资源中 3 个预留给正在等待的线程，2 个可以被直接获取

(续)

操作 (按时间排序)	持有线程	等待线程	value	wakeup	剩余资源	注 释
新来 2 个线程请求资源	7	3	0	3	3	直接消耗 value, 无须等待
1 个等待线程获取资源	8	2	0	2	2	等待线程消耗 wakeup
新来 1 个线程请求资源	8	3	-1	2	2	虽然 wakeup 不为 0, 但已经预留给之前等待的线程。value 再次被减到负数, 新的线程必须等待
1 个线程释放资源	7	3	0	3	3	
1 个线程释放资源	6	3	1	3	4	此时等待的线程都已经预留对应资源, 释放的资源直接加到 value
6 个线程释放资源	0	3	7	3	7	现在没有线程拿着资源
3 个等待线程获取资源	3	0	7	0	10	等待线程消耗 wakeup
3 个线程释放资源	0	0	10	0	10	所有资源被释放

下面将详细分析信号量的两个操作的实现。首先分析信号量的 wait 操作。线程获取该信号量的互斥锁后 (第 10 行), 先将 value 的值减 1 (第 11 行)。若其值大于等于 0, 则代表有空闲的资源, wait 操作成功。如果 value 的值小于 0, 那么它不再表示当前可用的资源数量, 我们需要检查 wakeup 来判断是否还有空闲的资源可以消耗。当 wakeup 大于 0 时, 代表还有空闲的资源可以消耗。而如果 wakeup 也为 0, 就需要用 10.2 节实现的 cond_wait 将当前线程挂起等待 (第 14 行)。

小思考

在 wait 操作中, 为何使用 do...while 而非 while? 有什么差别吗?

这里使用 do...while 而非 while, 是为了提供有限等待的保证, 确保调用 signal 后立刻调用 wait 的线程不会直接拿走资源, 而是交给正在等待的线程。下面通过一个例子进行详细说明。

假设线程 0 与线程 1 两个线程使用信号量进行同步, 它们的执行顺序如表 10.3 所示。在 T_0 时刻, 信号量的 value 为 0。此时线程 0 调用 wait 操作挂起等待。随后在 T_1 时刻, 线程 1 调用 signal 操作给 wakeup 加 1, 表明此时有可用资源, 并尝试唤醒线程 0。由于调用 wakeup 的时间与线程 0 被调度执行有一定的时间差, 线程 0 在 T_3 时刻才真正被唤醒。在 T_2 时刻, 线程 1 再次调用 wait。在使用 while 的情况下, 线程 1 将可能在线程 0 被唤醒前 (T_3) 抢先一步拿到该资源。如此往复, 线程 1 可能会一直自己生产、消耗信号量, 而线程 0 一直无法执行。因此, 实现中使用 do...while, 要求线程在检查 wakeup 之前调用 cond_wait 并排队, 避免造成线程 0 一直无法被唤醒。

表 10.3 使用信号量的某一执行顺序

时刻	线程 0	线程 1
T_0	wait(&S) 挂起等待	

(续)

时刻	线程 0	线程 1
T_1		signal(&S)
T_2		wait(&S) 拿到资源
T_3	被唤醒，发现没有可用资源	

对应地，在执行 signal 操作时，线程首先将 value 的值加 1（第 24 行），然后判断是否还有线程等待在该信号量上（第 25 行）。如果有则增加 wakeup（第 26 行），并使用 cond_signal 唤醒其中的一个线程获取资源（第 27 行）。

读者可以思考一下，代码片段 10.10 的实现能够保证按照调用 wait 的顺序拿到资源吗？实际上，以上实现并不能保证线程按照调用 wait 的顺序依次拿到资源。考虑这样的情况，系统中存在线程 0、线程 1 与线程 2，它们的执行顺序如表 10.4 所示。在 T_0 时刻，信号量的 value 为 0。此时，线程 0 调用 wait 操作挂起等待。在 T_1 与 T_2 时刻，线程 1 与线程 2 分别调用 signal 操作增加可用资源。此时信号量的 value 为 1，而 wakeup 也为 1。在 T_3 时刻，线程 1 调用 wait 操作后，无须等待便可直接拿到资源。线程 0 则到稍晚的 T_4 时刻被唤醒之后才能拿到资源。上述例子中，线程 0 虽然比线程 1 更早调用 wait 操作，却更晚拿到资源。但这并不会破坏有限等待的保证，出现上述情况的前提是信号量此时能够满足所有挂起线程的使用，value 才有可能被加回正数。因此即使后来的线程可能抢先一步拿到资源，但是挂起的线程也能够被唤醒后拿到资源。所以不会出现阻塞挂起线程的情况。

表 10.4 使用信号量的三个线程的执行顺序

时刻	线程 0	线程 1	线程 2
T_0	wait(&S) 挂起等待		
T_1		signal(&S)	
T_2			signal(&S)
T_3		wait(&S) 拿到资源	
T_4	被唤醒，拿到资源		

10.4 读写锁的实现

本节主要知识点

- ❑ 如何正确实现偏向读者的读写锁？
- ❑ 如何正确实现偏向写者的读写锁？